

Edge Intelligence LAB

Lab Task 2 : Analyzing MNIST- Image Dataset using CNN

Ashish Vikas Nikam

25MML0015

Jupyter Code For lab task : https://colab.research.google.com/drive/1H-Xm41gSetKGnh7HZduKbhSNomu-siES?usp=drive_link

GitHub Repository link :

https://github.com/ashish222001/25MML0015_Ashish.Nikam_MACSE604_EdgeLab01

Code :

```
[ ] import os
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout

[ ] import tensorflow as tf
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()

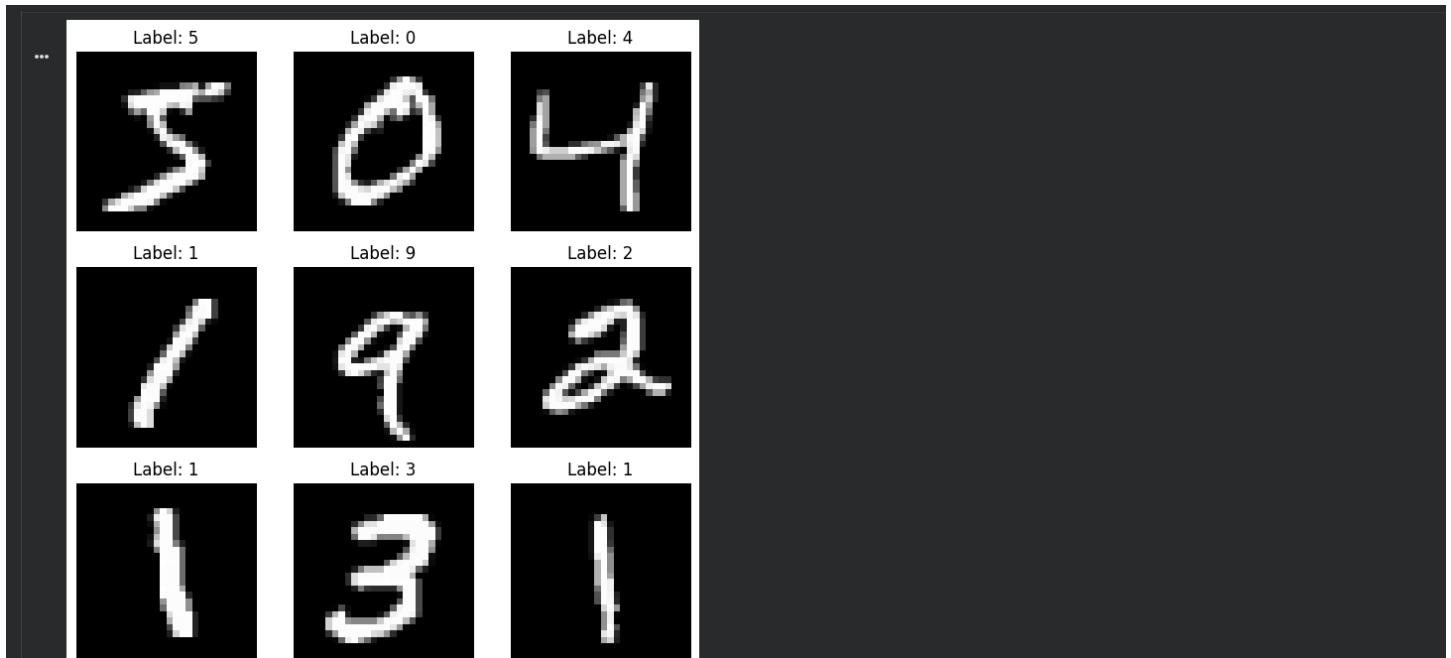
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 2s 0us/step

[ ] print(x_train.shape) # (60000, 28, 28)
print(x_test.shape) # (10000, 28, 28)

(60000, 28, 28)
(10000, 28, 28)

[ ] import matplotlib.pyplot as plt
plt.figure(figsize=(8,8))
for i in range(9):
    plt.subplot(3,3,i+1)
    plt.imshow(x_train[i], cmap='gray')
    plt.title(f"Label: {y_train[i]}")
    plt.axis("off")
plt.show()
```

Corresponding output :



```
[ ] x_train = x_train / 255.0
    x_test = x_test / 255.0

    x_train = x_train.reshape(-1, 28, 28, 1)
    x_test = x_test.reshape(-1, 28, 28, 1)

[ ] y_train = tf.keras.utils.to_categorical(y_train, 10)
    y_test = tf.keras.utils.to_categorical(y_test, 10)

[ ] from tensorflow.keras.models import Sequential
    from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout

    model = Sequential([
        Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)),
        MaxPooling2D(2,2),

        Conv2D(64, (3,3), activation='relu'),
        MaxPooling2D(2,2),

        Flatten(),
        Dense(128, activation='relu'),
        Dropout(0.5),
        Dense(10, activation='softmax')
    ])

... /usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input_shape`/'input_dim` argument to a layer. When using Sequential
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
[ ] ▶ model.summary()
```

```
... Model: "sequential"
```

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_1 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)	0
flatten (Flatten)	(None, 1600)	0
dense (Dense)	(None, 128)	204,928
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 10)	1,290

Total params: 225,834 (879.04 KB)
 Trainable params: 225,834 (879.04 KB)
 Non-trainable params: 0 (0.00 B)

```
[ ] model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)
```

```
[ ] ▶ history = model.fit(
    x_train,
    y_train,
    epochs=10,
    batch_size=32,
    validation_data=(x_test, y_test)
)
```

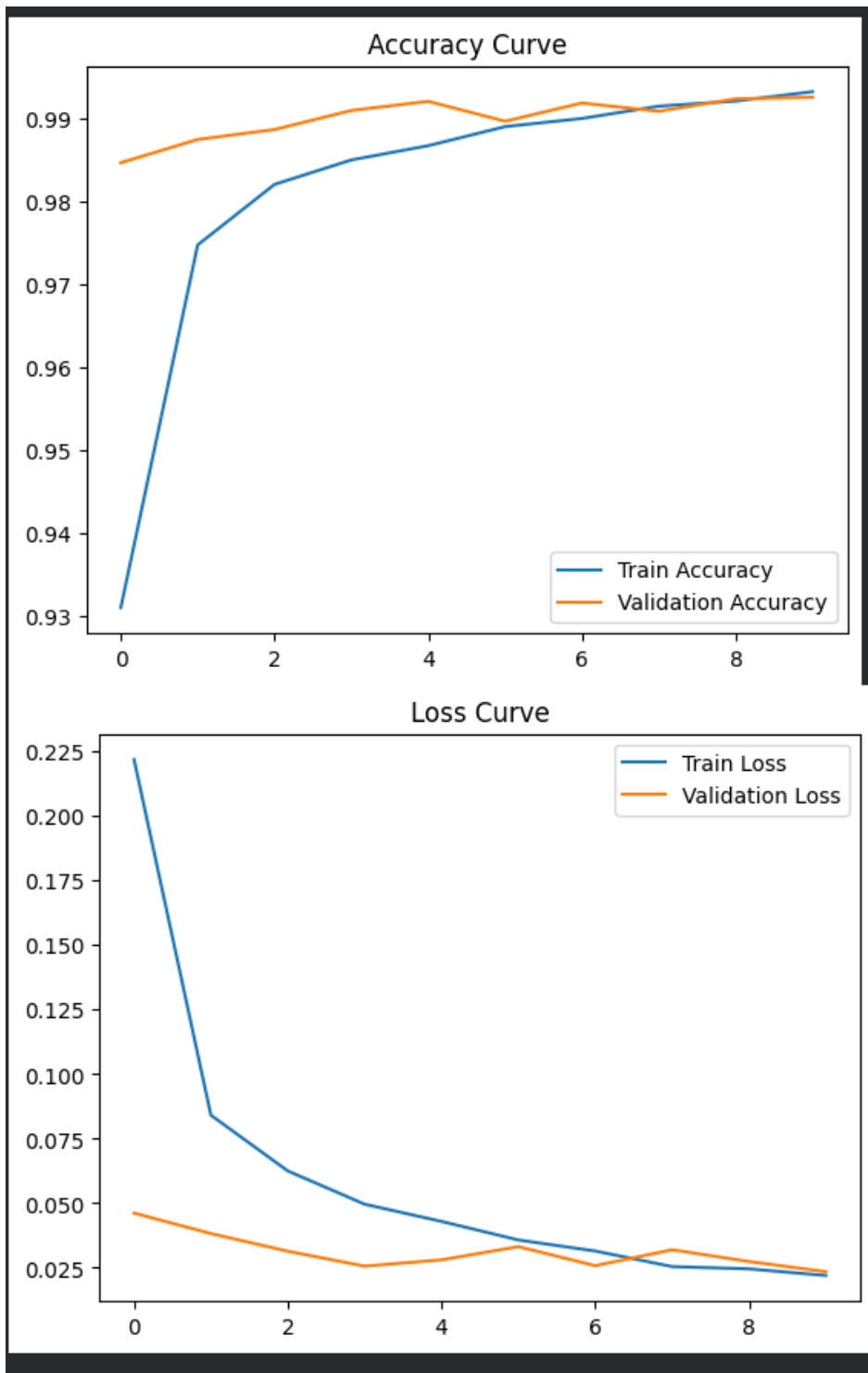
```
... Epoch 1/10
1875/1875 ————— 13s 4ms/step - accuracy: 0.8564 - loss: 0.4434 - val_accuracy: 0.9847 - val_loss: 0.0462
Epoch 2/10
1875/1875 ————— 7s 4ms/step - accuracy: 0.9739 - loss: 0.0858 - val_accuracy: 0.9875 - val_loss: 0.0383
Epoch 3/10
1875/1875 ————— 7s 4ms/step - accuracy: 0.9817 - loss: 0.0635 - val_accuracy: 0.9887 - val_loss: 0.0314
Epoch 4/10
1875/1875 ————— 7s 4ms/step - accuracy: 0.9840 - loss: 0.0531 - val_accuracy: 0.9910 - val_loss: 0.0256
Epoch 5/10
1875/1875 ————— 7s 3ms/step - accuracy: 0.9875 - loss: 0.0408 - val_accuracy: 0.9921 - val_loss: 0.0281
Epoch 6/10
1875/1875 ————— 7s 4ms/step - accuracy: 0.9895 - loss: 0.0335 - val_accuracy: 0.9897 - val_loss: 0.0332
Epoch 7/10
1875/1875 ————— 7s 4ms/step - accuracy: 0.9898 - loss: 0.0312 - val_accuracy: 0.9919 - val_loss: 0.0258
Epoch 8/10
1875/1875 ————— 7s 4ms/step - accuracy: 0.9920 - loss: 0.0238 - val_accuracy: 0.9909 - val_loss: 0.0320
Epoch 9/10
1875/1875 ————— 7s 4ms/step - accuracy: 0.9931 - loss: 0.0216 - val_accuracy: 0.9924 - val_loss: 0.0274
Epoch 10/10
1875/1875 ————— 10s 3ms/step - accuracy: 0.9940 - loss: 0.0198 - val_accuracy: 0.9926 - val_loss: 0.0235
```

```
[ ] test_loss, test_accuracy = model.evaluate(x_test, y_test)
print("Test Accuracy:", test_accuracy)

313/313 ————— 1s 2ms/step - accuracy: 0.9903 - loss: 0.0306
Test Accuracy: 0.9926000237464905
```

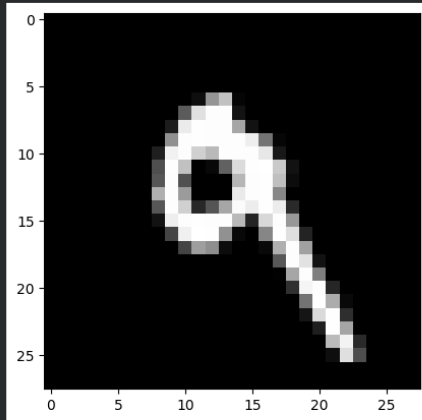
```
[ ] ▶ plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.legend()
plt.title("Accuracy Curve")
plt.show()

plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.legend()
plt.title("Loss Curve")
plt.show()
```



```
[ ] import numpy as np
index = 7
plt.imshow(x_test[index].reshape(28,28), cmap='gray')
plt.show()

prediction = model.predict(x_test[index].reshape(1,28,28,1))
print("Predicted Digit:", np.argmax(prediction))
```



1/1 0s 32ms/step
Predicted Digit: 9

Saving the model using Pickle :

```
[ ] import pickle
import os

[ ] with open("mnist_cnn_model.pkl", "wb") as file:
    pickle.dump(model, file)

[ ] model_size = os.path.getsize("mnist_cnn_model.pkl") / (1024 * 1024)
print(f"Model size: {model_size:.2f} MB")

Model size: 2.61 MB

[ ] with open("mnist_cnn_model.pkl", "rb") as file:
    loaded_model = pickle.load(file)

[ ] loaded_model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

[ ] loss, accuracy = loaded_model.evaluate(x_test, y_test)
print("Loaded Model Accuracy:", accuracy)

313/313 2s 4ms/step - accuracy: 0.9903 - loss: 0.0306
Loaded Model Accuracy: 0.992600237464905
```

Model Size : 2.61 MB

```
[ ] original_weights = model.get_weights()

[ ] loaded_weights = loaded_model.get_weights()

[ ] ▶ same = True
    for w1, w2 in zip(original_weights, loaded_weights):
        if not np.array_equal(w1, w2):
            same = False
            break

    print("Are weights identical?", same)

... Are weights identical? True

[ ] loaded_model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_1 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)	0
flatten (Flatten)	(None, 1600)	0
dense (Dense)	(None, 128)	204,928
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 10)	1,290

Total params: 225,034 (879.04 KB)
 Trainable params: 225,034 (879.04 KB)
 Non-trainable params: 0 (0.00 B)

```
[ ] ▶ index = 10
    prediction = loaded_model.predict(x_test[index].reshape(1,28,28,1))
    print("Predicted Digit:", np.argmax(prediction))

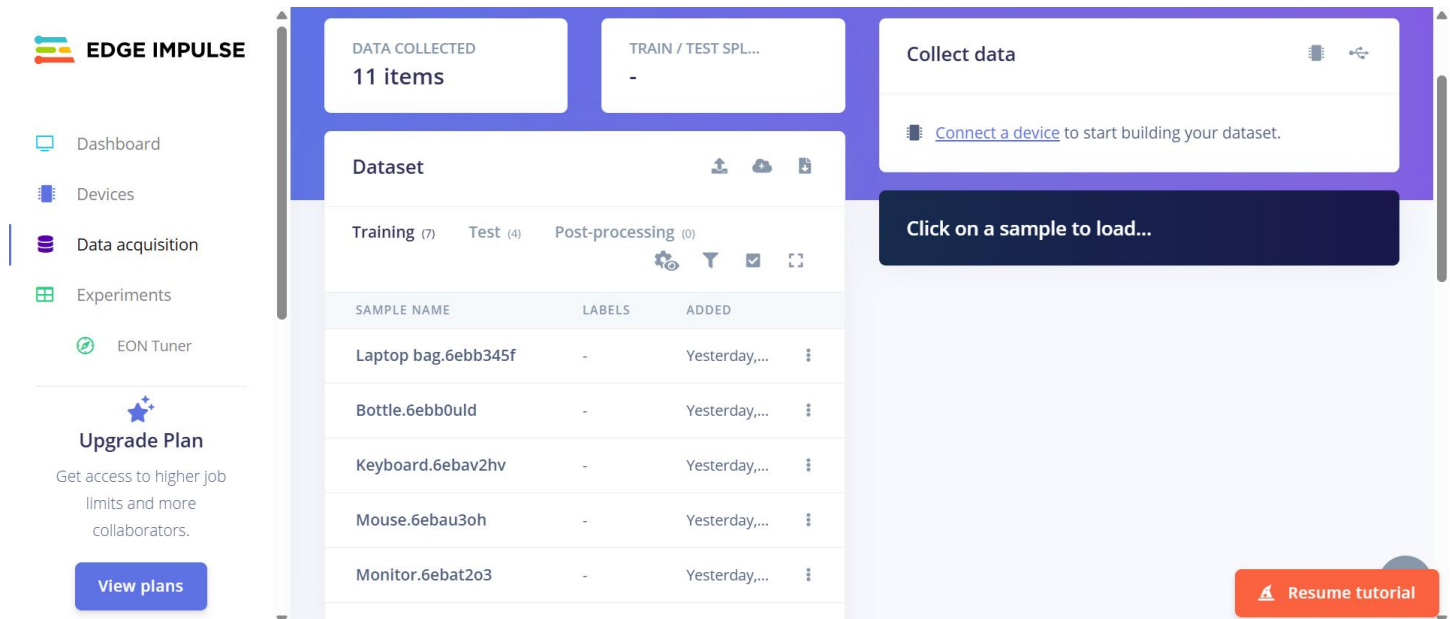
... 1/1 1s 542ms/step
    Predicted Digit: 0
```

Part 2 :

1. Create account in edge impulse :

The screenshot displays the Edge Impulse web interface. On the left is a sidebar with navigation links: Dashboard, Devices, Data acquisition, Experiments, EON Tuner, and an Upgrade Plan section. The main content area shows the project 'AshishNikam-project-1' under the 'PERSONAL' tab. The project target is set to 'Cortex-M4F 80MHz'. Below the project name, there's a description: 'This is your Edge Impulse project. From here you acquire new training data, design impulses and train models.' A 'New tag' button is visible. The interface is divided into sections: 'Getting started' with instructions to build a dataset or validate model performance, and 'Sharing' with a 'Private' dropdown. A 'Resume tutorial' button is in the bottom right corner.

2. Go to data acquisition



EDGE IMPULSE

Dashboard
Devices
Data acquisition
Experiments
EON Tuner

Upgrade Plan
Get access to higher job limits and more collaborators.
[View plans](#)

DATA COLLECTED
11 items

TRAIN / TEST SPL...
-

Collect data
[Connect a device](#) to start building your dataset.

Dataset

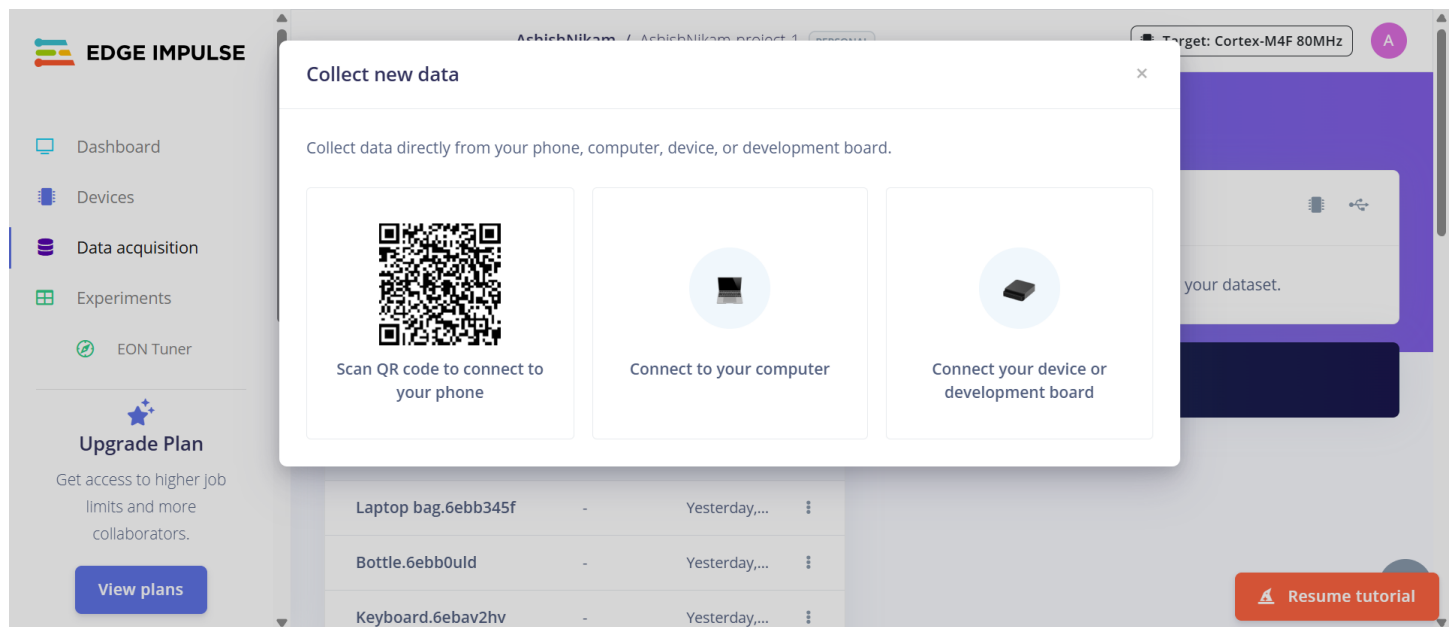
Training (7) Test (4) Post-processing (0)

SAMPLE NAME	LABELS	ADDED
Laptop bag.6ebb345f	-	Yesterday,...
Bottle.6ebb0uld	-	Yesterday,...
Keyboard.6ebav2hv	-	Yesterday,...
Mouse.6ebau3oh	-	Yesterday,...
Monitor.6ebat2o3	-	Yesterday,...

Click on a sample to load...

[Resume tutorial](#)

3. Choose connect data option and Scan QR using phone



EDGE IMPULSE

Dashboard
Devices
Data acquisition
Experiments
EON Tuner

Upgrade Plan
Get access to higher job limits and more collaborators.
[View plans](#)

Collect new data

Collect data directly from your phone, computer, device, or development board.

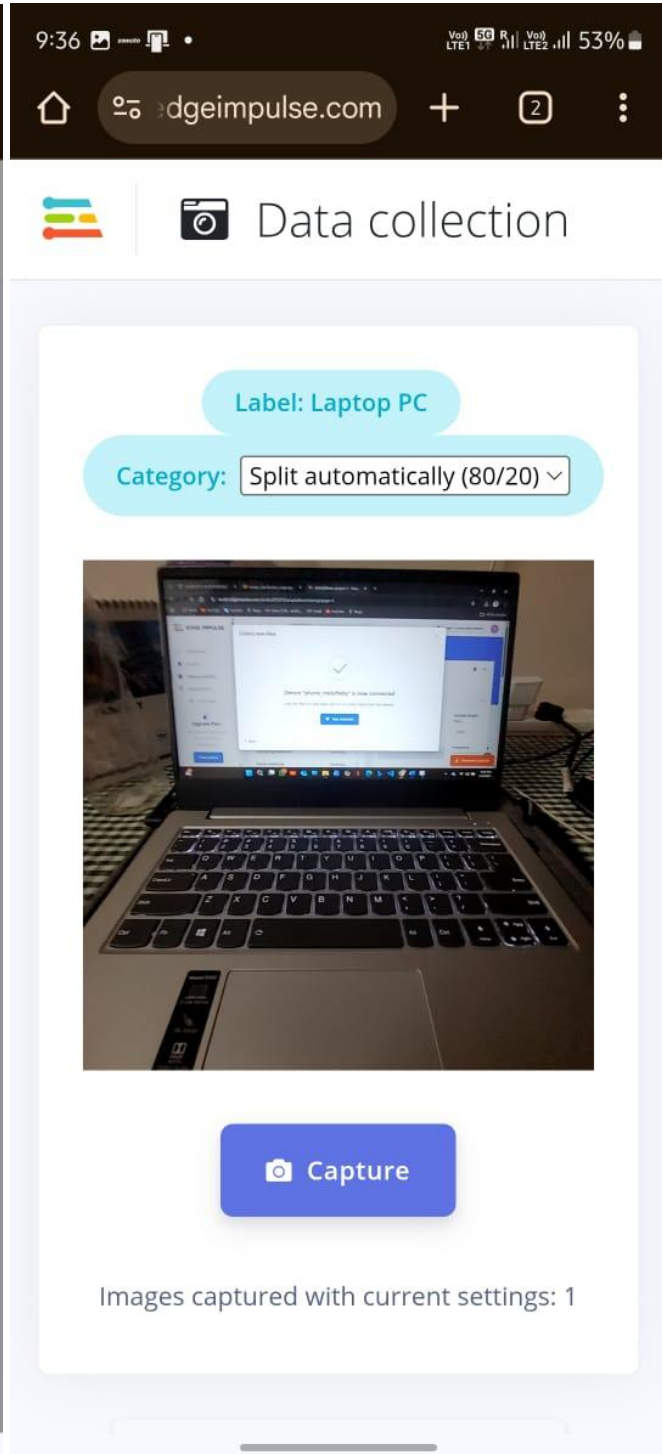
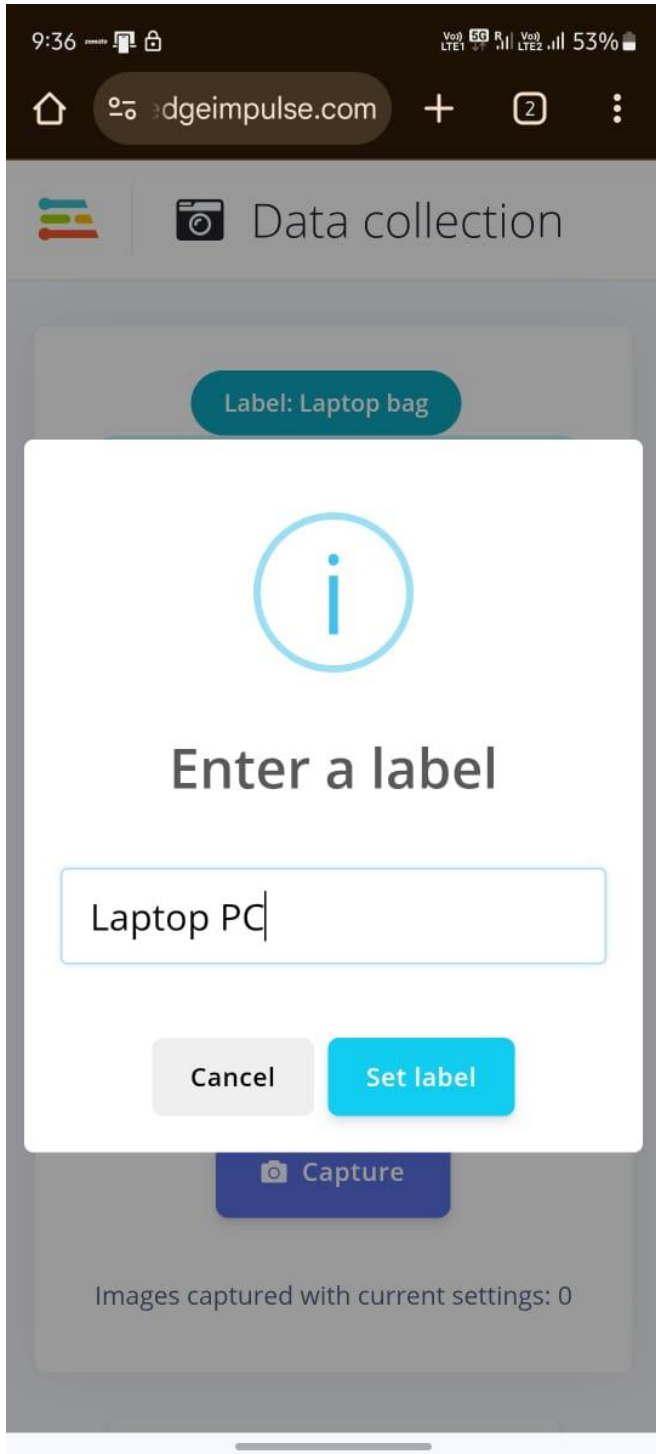

Scan QR code to connect to your phone


Connect to your computer


Connect your device or development board

Target: Cortex-M4F 80MHz

[Resume tutorial](#)

4. Select Label data before clicking a photo.

5. Split the clicked photos into train and test data

 **EDGE IMPULSE**

[Dashboard](#)
[Devices](#)
[Data acquisition](#)
[Experiments](#)
[EON Tuner](#)


Upgrade Plan
Get access to higher job limits and more collaborators.
[View plans](#)

DATA COLLECTED
12 items

TRAIN / TEST SPL...
-

Dataset

Training (8) Test (4) Post-processing (0)

SAMPLE NAME	LABELS	ADDED
Laptop PC.6eecp676	-	Today, 21:...
Laptop bag.6ebb345f	-	Yesterday,...
Bottle.6ebb0uld	-	Yesterday,...
Keyboard.6ebav2hv	-	Yesterday,...
Mouse.6ebau3oh	-	Yesterday,...
Monitor.6ebat2o3	-	Yesterday,...

Collect data

[Connect a device](#) to start building your dataset.

Click on a sample to load...

[Resume tutorial](#)